Auditing Actual Execution and Output Portability in Quantized LLM Inference: A Measurement Study of llama.cpp

Anonymous

October 2026 (technical note)

Abstract

We measured whether the same quantized GGUF model, the same prompt, and greedy decoding (temperature 0) produce the same output across the compute backends of llama.cpp (release b11327, official binaries, SHA256-recorded) on a 12-machine multi-vendor fleet (NVIDIA, AMD, Intel; Linux and Windows). Auditing *which device actually executed each run* — screening every (machine, backend) configuration via per-prompt wall-time ratios against the same-box CPU build, with direct on-box corroboration (process maps, VRAM readings, library-load checks) for a subset — turned out to be the decisive methodological step: three distinct failure classes silently served inference from the CPU while presenting as GPU backends — no GPU-related console output appeared in any diagnosed fallback run — including the official ROCm build on four AMD hosts (its backend library requires ROCm userspace SONAMEs one major version newer than the stock ROCm 6.3.0 installation on the host we diagnosed directly, and the release bundles no Linux ROCm runtime), a CUDA-13 build deprived of its runtime libraries by our own launch environment, and a virtualized host without access to its advertised NVIDIA GPU. On runs classified as GPU-executed by this audit, byte-exact agreement with the same-box CPU reference is quantization-dominated and monotone — F16 88.0%, Q8_0 35.3%, Q4_K_M 18.0% of 8,670 prompt comparisons (34 raw-completion prompts per cell, 128-token cap) — and the same precision ordering appears between *the GPU builds' shared CPU implementation* and the standalone CPU build on the same host (F16 92.7%, Q8_0 40.5%, Q4_K_M 17.7%) and between the same CPU build on hosts from different CPU equivalence classes. Divergence is concentrated at near-ties: among the 7,875 first divergences, the alternative token lies within the reference's recorded top-5 in 99.89% of events (median logprob gap 0.062 nats, p90 0.231). Both identical-stack stability controls reproduced every tested generation: 578/578 comparisons across two physical machines with identical GPU/driver/binary, and 1,734/1,734 in a same-machine 12-hour re-run of 51 cells. We release the harness, per-token logprob records, and the device-truth audit method. Practical conclusions: verify actual execution placement before labeling any measurement with a backend name; do not assume exact-output portability for quantized artifacts when changing the numerical implementation at Q4-class precision; and do not use default-tool perplexity as a conformance signal.

1 Introduction

A greedy-decoded output is usually treated as a property of the model and the prompt. This note reports a measurement study of the other variable: the numerical stack that executes the model. We ask two questions. First, *does the intended backend actually execute the workload*? Second, *when the stack differs, how much does greedy output differ, and where*?

The first question became the pivot of the study. We began by comparing backends on a multi-vendor fleet and initially read the results as backend behavior. A device-truth audit — comparing every run's per-prompt generation wall time against the same-box CPU build — revealed that three of our "GPU backend" columns had served from the CPU without a fatal error: the official ROCm build on all four AMD hosts, the CUDA-13 build on both hosts offering it, and CUDA on a virtualized host. Only the first involves the upstream binary's runtime requirements; the second was a launch-environment mistake in our own harness (which had passed the runtime-library path to device enumeration but not to the server process); the third was a virtual machine without access to its advertised NVIDIA GPU. All three served HTTP requests normally; in every run we diagnosed directly the console carried no GPU-related output at all — no error, no warning, no offload summary — and none can have appeared in the sweep runs either, since their servers were never passed `-ngl`, the warning's only trigger. The silence is structural, not incidental: release builds suppress backend-load failures entirely, and the engine's "no usable GPU found" warning is printed only when `-ngl` is passed explicitly — and even then suppressed whenever the bundled RPC backend is registered, as it was in every run we diagnosed. ROCm device enumeration returned no device; in the CUDA-13 case, enumeration succeeded under a different environment from the server and therefore did not establish server offload. Enumeration, we conclude, does not by itself establish execution.

The audit changed the study's claims in both directions. It led us to withdraw findings we had briefly believed (a "CUDA-toolkit changes everything" comparison; a "perplexity defaults differ between tools" hypothesis): the compared processes had not run under equivalent runtime environments or execution placements, and several of the comparisons had in fact measured the GPU builds' shared CPU implementation against the standalone CPU build. It also sharpened the findings that survive: the conformance structure of runs *classified* by the audit as differing in execution, the near-tie anatomy of divergence, and the complete agreement of our identical-stack controls.

2 Method

Fleet. 12 machines, one of them a virtualized guest (13 run labels; one machine was measured twice, 12 hours apart; the re-run is excluded from the primary conformance table, used as a stability control, and included in the full divergence-event analysis): NVIDIA (RTX 5060 Ti, RTX PRO 6000 Blackwell, TITAN RTX, RTX 3060 Ti/Windows, and a virtualized RTX 4090 guest that turned out to lack access to its advertised NVIDIA GPU — §3), AMD (RX 7900 XTX ×2, Radeon AI PRO R9700, Strix Halo 8060S), Intel (Arc A770 Linux + Windows, Arc B60); hosts span several CPU generations and both desktop OSes.

Engine and models. llama.cpp release b11327, official prebuilt binaries only, SHA256 recorded. 17 core GGUF model files: Qwen2.5-Instruct 0.5B/1.5B/3B and SmolLM2-Instruct 135M/360M/1.7B at Q4_K_M and Q8_0, plus F16/FP16 for the five models up to 1.7B — the 3B was not run at F16 — with official Qwen-org quantizations and community re-quantizations for SmolLM2 (provenance is a caveat for family comparisons).

Protocol. 34 fixed raw-completion prompts (no chat template; four categories), greedy decoding (temperature 0), 128-token cap, prompt caching disabled, one request at a time. The server protocol records, for every generated token on *every* backend, the chosen id, its logprob, and the top-5 ids with logprobs. Perplexity is measured by the same release's `llama-perplexity` on a fixed ∼5k-token corpus. The reference for all per-cell comparisons is the same host running the release's standalone CPU build (one binary on every Linux host; the Windows CPU build on the two Windows hosts).

Device-truth audit. For every (machine, backend) configuration we compute the median

over models of the ratio of median per-prompt generation wall time to the same-box CPU build's. Observed ratios bifurcate cleanly in our data (GPU-classified configurations at 0.06–0.66, fallback configurations at 1.00–1.03). We corroborated the classification directly on-box for one configuration per failure class — process maps, VRAM readings, and library-load checks (§3) — plus one genuine-GPU CUDA configuration and three pinned Vulkan re-runs; the remaining classifications rely on the observed timing separation, corroborated by device listings in the later supplemental records (the core records predate that field). A configuration at ≈ 1.0 is classified *CPU-fallback* and analyzed as a separate population, not as a GPU backend.

3 Finding 1: silent CPU fallback

Three failure classes, one user-visible behavior: the server accepts requests and generates normally, and in every fallback run we diagnosed the console carried no GPU-related output at all — no error, no warning, no offload summary, not even with an explicit `-ngl 999`. The engine's source at this release explains the silence: release builds load backends with failure messages suppressed; the "no usable GPU found" warning in `common/arg.cpp` fires only inside the `-ngl` option handler; and even there it is guarded by `llama_supports_gpu_offload()`, which returns true whenever the bundled RPC backend is registered — as it was in every run we diagnosed (its library appears in the process maps). The process serves on the CPU regardless.

(a) Missing backend userspace (upstream-relevant). The official ROCm release asset (named `rocm-10.0`) ships a `libggml-hip.so` that requires `libamdhip64.so.7`, `libhipblas.so.3`, and `librocblas.so.5`; the directly-diagnosed host, running the stock ROCm 6.3.0 userspace, provides `.so.6/.so.2/.so.4`; and the release bundles no Linux ROCm runtime (a Windows runtime zip exists; the Linux equivalent does not). On the directly diagnosed host (an RX 7900 XTX) the backend library never loads; on the other three AMD hosts the same failure follows from the CPU-class wall-time signature, printed-precision perplexity equality, and mutually bit-identical fallback outputs. On all four, `--list-devices` prints `(none)` (recorded per-record in the supplemental runs), and the server runs fully on the CPU, *including with an explicit `-ngl 999`*: process maps show no HIP library, VRAM stays idle, and generation proceeds at approximately the standalone CPU build's speed. The perplexity tool is subject to the same backend failure, so its values are host-CPU values; on our data the "ROCm" perplexity matched the CPU build's in 68 of 68 cells (at the tool's printed precision). The "ROCm" generations, however, did *not* match the standalone CPU build's byte-for-byte (46.8% pooled): the fallback runs executed the GPU builds' bundled CPU implementation, which is shared across the ROCm, CUDA, and CUDA-13 release builds and differs from the standalone CPU build's. The fallback outputs are bit-identical across the three AMD hosts sharing a host-CPU class (578/578 pairwise), consistent with same-build, same-CPU-class execution. Related reports exist upstream for Windows and other setups[1]; our contribution is the Linux SONAME detail, the cross-host confirmation, and the benchmarking consequence: quality measurements taken this way measure *a* CPU implementation — the fallback build's own, not the standalone CPU build's — while presenting as GPU measurements.

(b) Runtime-library environment mistakes (our own). Our harness passed the CUDA runtime-library path (`LD_LIBRARY_PATH`) to device enumeration and to the perplexity tool, but not to the server process. On hosts where `libcudart.so.13` exists only in the release's runtime tarball, the CUDA-13 backend loaded for `--list-devices` — which therefore reported the GPU — but not for the server, which served from the CPU without a fatal error. An on-box A/B check isolates the effect at matched settings (`-ngl 999` both times): with the environment set, the CUDA-13 build

[1] ggml-org/llama.cpp issues #26964, #22748, #26929, #27923.

offloads at 602 tok/s with 456 MiB VRAM in use; without it, it runs on the CPU at 203 tok/s with 2 MiB VRAM in use. On the same host, the CUDA-12.8 build was unaffected because the system carries CUDA 12.9 libraries. We retract the preliminary "CUDA-13 behaves like a different backend" observation that this artifact produced, and note the general lesson: `--list-devices` output reflects the environment of whatever process ran it, which is not necessarily the environment of the server.

(c) Absent hardware. One host, labeled as a virtualized RTX 4090, has no NVIDIA device on its guest PCI bus at all (no `/dev/nvidia*`, no driver library, only a guest-visible Intel iGPU on `lspci`). Its CUDA runs are classified as CPU-executed by the audit; its Vulkan runs are excluded from the study (they enumerated and used the guest-visible Intel iGPU). We retain this host's CUDA cells only within the CPU-fallback population and note that our earlier "virtualized 4090" hardware label was wrong.

Detection. None of the three classes produced any GPU-related console output, even with an explicit `-ngl` — the registered RPC backend suppresses the engine's only GPU-availability warning, and release builds swallow backend-load failures silently. Enumeration correctly reported no device when the ROCm backend could not load; in the CUDA-13 case it reported the GPU — from a process running in a different environment than the server, so that result did not establish server offload. The wall-time-ratio audit caught all three, and it is cheap: it requires only a CPU-build baseline on the same host. We recommend it as a standing screening check in any benchmarking or conformance setup, followed by post-launch verification that the backend library is actually mapped in the server process.

4 Finding 2: conformance on runs classified as GPU-executed

On runs classified as GPU-executed by the audit (CUDA on 4 machines — the 12.8 build on three Linux hosts and the 12.4 build on the Windows host — Vulkan on 11; 255 cells after removing the re-run; 34 raw-completion prompts per cell with a 128-token cap), byte-exact agreement with the same-box CPU reference is:

Population	Q4_K_M	Q8_0	F16/FP16
GPU vs same-box CPU (n: 3060/3060/2550)	18.0%	35.3%	88.0%
GPU builds' shared CPU impl. vs standalone CPU (n: 1428/1428/1190)	17.7%	40.5%	92.7%
Same CPU build, same host-CPU class (22 host pairs)	100%	100%	100%
Same CPU build, different host-CPU class (23 pairs)	13.7–20.1%	35.3–39.2%	90.0–94.1%

Denominators are prompt comparisons. F16/FP16 covers five model files and Q4_K_M/Q8_0 cover six (Qwen2.5-3B was not run at F16); cross-host rows compare 578 model–prompt combinations per pair (204 at Q4, 204 at Q8, 170 at F16). The second row's 4,046 comparisons collapse to 1,734 distinct (host-class, model, prompt) combinations — 1,269 distinct content pairs — because the GPU builds ship one shared CPU implementation, so fallback runs on hosts sharing a CPU class are bit-identical to each other. Equivalence classes are empirical — groups of hosts whose standalone CPU-build outputs are identical across all 578 combinations (class A: seven Linux hosts, including the TITAN RTX and PRO 6000; class B: the second 7900 XTX host and the 4090 guest; the 5060 Ti host is a singleton). The classes track CPU instruction-set tiers — class A hosts expose AVX-512 without AMX, class B hosts lack AVX-512, and the singleton carries AMX — and are coarser than kernel-variant names: the two directly-mapped class-B hosts loaded different AVX2-tier CPU variants with identical outputs. Cross-host rows use the Linux CPU binary only; the two Windows hosts agree with each other under the Windows CPU build and are excluded from these rows.

Three observations. First, among the GPU/CPU, cross-build CPU, and cross-class CPU comparisons, agreement increased strongly with precision, and the pooled "∼45–48% agreement" that a fixed quant mix produces is a mixture artifact — per-precision rates span approximately 14% to 94% in these populations; in contrast, all tested same-class CPU host pairs agreed completely at every precision. Second, the comparison type is a second-order effect: GPU-vs-CPU, the GPU builds' shared CPU implementation vs the standalone CPU build's (including the ROCm and CUDA-13 fallback columns, reclassified), and cross-class CPU-vs-CPU all follow the same precision ordering. Excepting same-class host moves, byte-exact portability across *the stack changes we tested* is largely absent at Q4-class precision and partially preserved at F16. Third, cross-host CPU comparisons are bimodal — every pair is either bit-identical or diverges like any other stack change, with nothing in between — and we group the identical pairs into empirical host-CPU equivalence classes (our Linux fleet has two nonsingleton classes and one singleton; the Windows pair also agrees internally). "Same result as CPU" therefore means "same result as this host's CPU build — and only hosts in its equivalence class".

5 Finding 3: agreement in identical-stack controls

Two stability controls bound the divergence findings. Two physically distinct machines with the same GPU (RX 7900 XTX), same OS, same kernel driver version (amdgpu 6.10.5; the Vulkan userspace version was not recorded), and same Vulkan binary produced bit-identical generations for 578/578 prompt comparisons. The same machine re-run 12 hours later (51 cells, 1,734 prompts; CUDA, CUDA-13, and Vulkan) reproduced its earlier outputs bit-for-bit in 100.0% of comparisons. These controls demonstrate repeatability in the tested configurations and support interpreting the observed differences as stack-dependent; they do not establish determinism for stacks we did not test. (We separately note that the engine's own documentation warns that prompt-cache reuse can introduce run-to-run nondeterminism via batch-shape-dependent prefill/decode arithmetic[2]; our protocol runs with caching disabled.)

6 Finding 4: divergence anatomy

Among the 7,875 first-divergence events (6,908 after removing the same-box re-run's 967 duplicate events; the figures below use the full set — deduplicated top-5 membership is 99.90% vs the full set's 99.89%), the diverging backend's chosen token lies within the reference's recorded top-5 in 99.89% of events. The exceptions are all in SmolLM2 Q4_K_M, at four (model, prompt) loci (0-based prompt indices): the 9 reference-side exceptions sit at two (135M prompt 23; 360M prompt 26), the 9 backend-side exceptions at three (360M prompt 26; 135M prompt 22; 360M prompt 1), and the two sets overlap in 5 events. The events span the primary per-cell comparison populations: 4,796 GPU-vs-CPU, 2,112 build-CPU-vs-CPU, and 967 from the re-run machine's cells against its own CPU reference. Each directional gap is available for 7,866 events (both gaps jointly for 7,862 of them); the quantiles below use each direction's available events separately. In the reference distribution, the preference for its own argmax over the backend's choice has median 0.062 nats (p90 0.231, p99 0.564); in the backend's distribution, the preference for its choice over the reference's has median 0.059 nats (p90 0.226). In 7,090 of the 7,875 events (90.0%) the divergence is a mutual rank-1↔2 swap — each side's chosen token is the other's runner-up. Divergence at greedy decoding is therefore concentrated at near-ties — compatible with the rounding-order mechanism assumed

[2] `tools/server/README.md` in the b11327 release tree.

in prior work [7], though our data does not by itself exclude small systematic implementation differences that would also preferentially flip low-margin decisions.

7 Related work

Cross-platform numerical deviation in DNN inference, with platform equivalence classes, was established for vision models by Schlögl et al. [1]. He (Thinking Machines Lab, 2025) [2] identifies batch-shape dependence as a source of temperature-0 serving nondeterminism and ships batch-invariant kernels; the engine's own cache-reuse warning is an instance of this class. Yuan et al. [3] measure greedy divergence across GPU type, count, batch size and numerical precision and propose LayerCast; Du et al. [7] measure cross-precision greedy divergence with margin-thresholded mitigation; Dutta et al. [4] show that aggregate accuracy metrics can hide flips in compressed models, and our fallback observations add a distinct failure mode: a perplexity tool and a generation server can run under different execution placements, so the aggregate metric may be computed by a different implementation than the one being evaluated. Pape et al. [5] quantify backend-choice effects across engines including llama.cpp. Deterministic or verified inference is addressed by Gond et al. [6], Cooper et al. [8] (fixed-reduction GEMMs), and TOPLOC (Ong et al. [9]) for verifiable inference. In software engineering, CRADLE [10] cross-validated deep-learning library backends; our harness follows that tradition for autoregressive LLM inference, and its device-truth audit extends it with an execution-placement check. Training-side tooling variance is covered by Zhuang et al. [11] and Pham et al. [12].

8 Limitations

One engine, one release (b11327) — rates may drift across releases, while the fallback classes and the audit method are the durable content. Results cover models from 135M to 3B; portability to larger models is unmeasured. No ROCm/HIP GPU execution was measured: AMD GPUs appear in the GPU-classified population only via Vulkan. Raw-completion prompts without chat templates may produce different output distributions from deployed chat prompting. The near-tie evidence is compatible with, but does not uniquely establish, rounding-order causation. Perplexity comparisons across backends are confounded by the fallback classes and by prefill-vs-decode kernel differences. SYCL contributed no complete cells (Linux binaries failed to load the oneAPI runtime; the Windows build loaded, demonstrably offloaded, then crashed mid-run).

9 Conclusion

For practitioners: (1) verify what executed — the wall-time ratio against a CPU baseline is a cheap screening check for silent CPU fallback, to be followed by execution-placement checks in the server process, and `--list-devices` output alone does not establish execution; (2) GPU-versus-local-CPU agreement was 18%, 35%, and 88% at Q4, Q8, and F16 respectively, with cross-build and cross-class CPU comparisons showing the same ordering — whereas same-class CPU host pairs agreed completely; do not assume exact-output portability when changing numerical implementations at Q4-class precision; (3) our identical-stack controls reproduced all tested outputs — pin backend, binary, GPU, and driver if you need reproducibility, and specify the complete execution configuration when claiming it; (4) do not treat default-tool perplexity as evidence of behavioral equivalence. We release the harness, the per-token logprob dataset, and the audit tooling at `https://github.com/shashb27/llamacpp-backend-conformance` (dataset: v1.0 release).

References

[1] A. Schlögl, N. Hofer, and R. Böhme. Causes and effects of unanticipated numerical deviations in neural network inference frameworks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[2] H. He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. Blog post, September 2025. `https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/`

[3] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu. Understanding and mitigating numerical sources of nondeterminism in LLM inference. *arXiv preprint* arXiv:2506.09501 (v2), 2025.

[4] A. Dutta, S. Krishnan, N. Kwatra, and R. Ramjee. Accuracy is not all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.09141.

[5] D. Pape, J. Evertz, and L. Schönherr. The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility. *arXiv preprint* arXiv:2605.19537, 2026.

[6] R. Gond, A. K. Kamath, R. Ramjee, and A. Panwar. LLM-42: Enabling determinism in LLM inference with verified speculation. *arXiv preprint* arXiv:2601.17768, 2026.

[7] G. Du, A. N. Khan, R. Zhou, X. Liu, D. Chakrabarti, F. Suya, and X. Li. Greedy decoding is not precision-invariant: Cross-precision output divergence in LLM inference. *Transactions on Machine Learning Research (TMLR)*, 2026. arXiv:2609.26621.

[8] L. Cooper, S. Jeong, H. Jeon, J. Young, and H. Kim. Accelerating the mitigation of LLM inference nondeterminism across GPU architectures. *arXiv preprint* arXiv:2609.25624, 2026.

[9] J. M. Ong, M. Di Ferrante, A. Pazdera, R. Garner, S. Jaghouar, M. Basra, M. Ryabinin, and J. Hagemann. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference. *arXiv preprint* arXiv:2501.16007, 2025.

[10] H. V. Pham, T. Lutellier, W. Qi, and L. Tan. CRADLE: Cross-backend validation to detect and localize bugs in deep learning libraries. In *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019. doi:10.1109/ICSE.2019.00107.

[11] D. Zhuang, X. Zhang, S. L. Song, and S. Hooker. Randomness in neural network training: Characterizing the impact of tooling. *arXiv preprint* arXiv:2106.11872, 2021.

[12] H. V. Pham, S. Qian, J. Wang, T. Lutellier, J. Rosenthal, L. Tan, Y. Yu, and N. Nagappan. Problems and opportunities in training deep learning software systems: An analysis of variance. In *IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 771–783, 2020. doi:10.1145/3324884.3416545.